Chapter 9: Linking to Real-world Data Sources and Services

Nature Inspired Optimisation for Delivery Problems (2022)

Lecture Notes — Self-contained Edition

Table of Contents I

9.1 Introduction — Why Real-world Data Matters I

Nature-inspired optimisation algorithms such as Ant Colony Optimisation, Genetic Algorithms, and Simulated Annealing are powerful mathematical tools. However, their value to practitioners depends critically on two things: obtaining *real* input data (actual road distances, not straight-line approximations) and presenting *results* in formats that other software systems can consume.

The Core Challenge

Most delivery optimisation algorithms expect, as input, a *distance matrix* — a table of travel distances or journey times between every pair of locations. In the real world, these distances depend on the road network, traffic conditions, and vehicle type. Computing them from geographic coordinates requires specialised tools.

This chapter introduces three families of tools that close the gap between abstract algorithms and real deployments:

1. **Geocoding** — converting human-readable addresses into latitude/longitude (lat/lon) coordinates.
2. **Routing services** — computing actual road-network distances and travel times between pairs of coordinates.

9.1 Introduction — Why Real-world Data Matters II

3. **Data export** — writing optimised routes in standard formats (KML, GPX, CSV) so they can be displayed in mapping software, loaded into GPS devices, or imported into business systems.

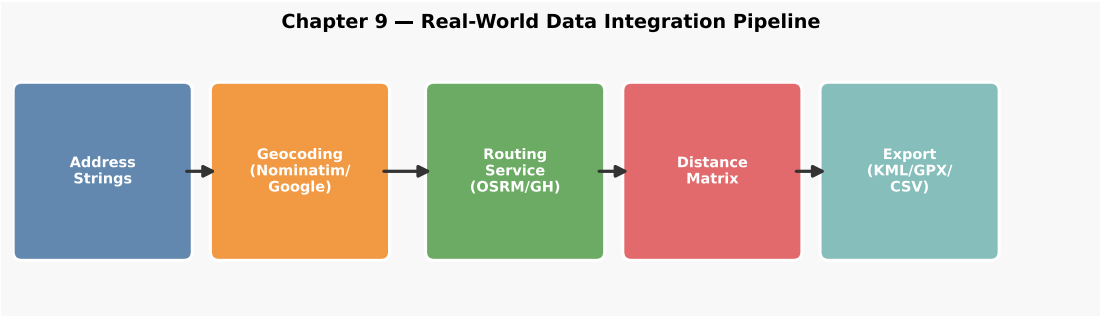
Key Insight

OpenStreetMap (OSM), launched around 2008, made high-quality geographic data freely available to any developer. Tools such as Nominatim (for geocoding) and GraphHopper (for routing) are built on top of OSM data, enabling sophisticated real-world solvers without commercial licensing costs.

9.1 Introduction — Why Real-world Data Matters III

The Overall Integration Pipeline

The diagram below shows how the components in this chapter fit together. A sequence of address strings is converted to coordinates by the geocoder, then a routing service computes the pairwise distance/duration matrix, the optimiser finds a good route, and finally an export service writes the result to a file format suitable for visualisation or navigation.



9.1 Introduction — Why Real-world Data Matters IV

Each component is designed as a *swappable module*: the book's Java code uses interfaces and abstract classes so that, for example, the Nominatim geocoder can be replaced by a Google Maps geocoder without touching the rest of the system. This design principle is equally applicable in Python.

9.2 Geocoding — Addresses to Coordinates I

What is geocoding?
Geocoding is the process of converting a human-readable location description — such as a postal address, a place name, or a postcode — into a pair of geographic coordinates: *latitude* (lat) and *longitude* (lon). These coordinates are numbers that pin a location precisely on the surface of the Earth.

Notation: Latitude and Longitude

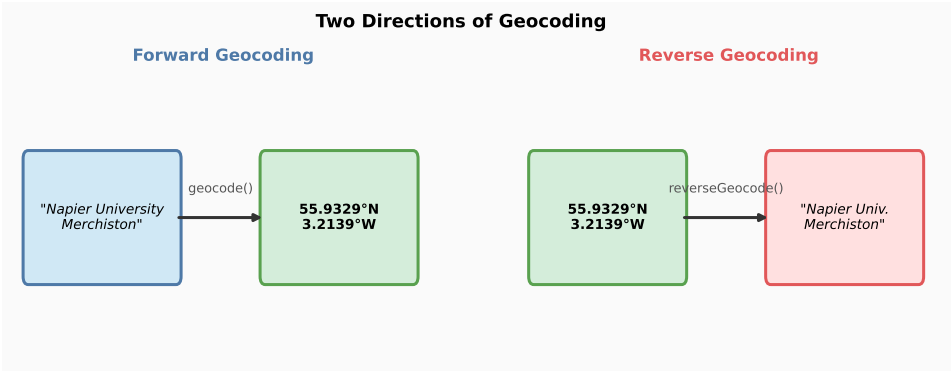
Latitude is measured in degrees from -90 (South Pole) to $+90$ (North Pole), where 0 is the Equator. A positive value means north of the Equator.
Longitude is measured in degrees from -180 to $+180$, where 0 is the Prime Meridian (Greenwich, UK). A negative value means west.
Example: Edinburgh Napier University, Merchiston Campus is at approximately lat $= 55.93$N, lon $= -3.21$W.

There are two directions of geocoding:

- **Forward geocoding**: given an address string, return coordinates. This is the most common need — a spreadsheet of delivery addresses must be converted to lat/lon before distances can be computed.

9.2 Geocoding — Addresses to Coordinates II

- **Reverse geocoding**: given coordinates, return the nearest address label. This is used when an optimiser returns a solution expressed in coordinates and a human-readable label is needed for the output.



9.2 Geocoding — Addresses to Coordinates III

Practical Considerations

A single address string can be ambiguous. For example, the string `"High Street, Edinburgh"` could match dozens of streets. A robust geocoder handles this by:

- Returning the most *important* match (e.g., the one with the highest population or the most prominent place type).
- Accepting structured input (street, city, postcode) to reduce ambiguity.
- Returning a *confidence score* so the application can flag uncertain results for human review.

For *reverse geocoding*, a coordinate may lie in the middle of a field far from any address. Two strategies exist:

1. Return the *nearest label*, regardless of distance. This always gives an answer but may be geographically misleading.
2. Return the nearest label only if it is *within a threshold distance* (e.g., 10 metres). If no label is close enough, report "not found".

The book implements both strategies; the second (threshold-based) approach is generally safer in production.

9.2 A Simple Geocoder — NapierLoc I

Before connecting to web services, the book introduces a *simple hand-crafted geocoder* called `NapierLoc`, which stores a hard-coded list of a few Edinburgh university campuses. While trivially small, it illustrates the key design patterns used throughout this chapter.

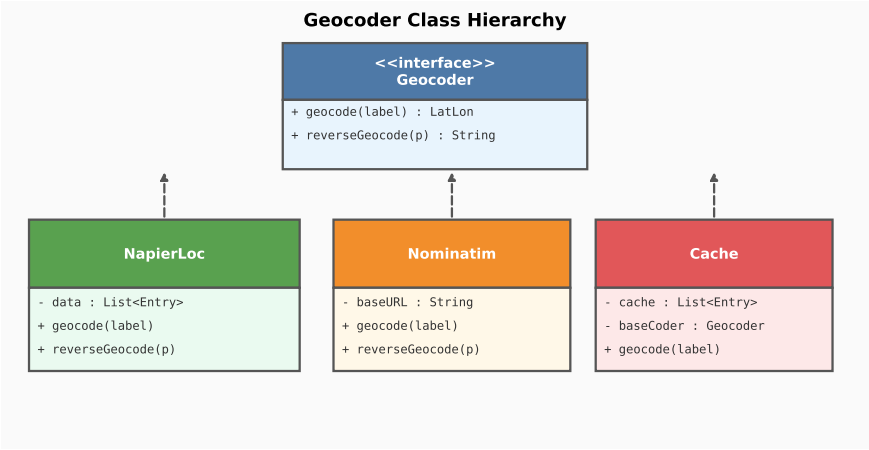
The Geocoder Interface (Java)

The book defines a Java `Geocoder` interface with exactly two methods:

- `geocode(String label)` — takes an address string, returns a `LatLon` object (latitude, longitude).
- `reverseGeocode(LatLon p)` — takes a `LatLon` point, returns the nearest label string.

All geocoders in the system implement this same interface. This means the optimisation algorithm never needs to know *which* geocoder it is using — it simply calls `geocode()` and gets a result.

9.2 A Simple Geocoder — NapierLoc II



9.2 A Simple Geocoder — NapierLoc III

How NapierLoc Works

The `NapierLoc` class stores a list of `Entry` objects. Each `Entry` holds one label–coordinate mapping, for example:

- `"Napier University, Merchiston"` $\rightarrow (55.9329, -3.2139)$
- `"Napier University, Craiglockhart"` $\rightarrow (55.9180, -3.2397)$
- `"Napier University, Sighthill"` $\rightarrow (55.9242, -3.2885)$

The `geocode()` method performs a *linear search* through the list looking for an exact label match. If found, it returns the associated coordinates; otherwise it returns `null`.
The `reverseGeocode()` method computes the *Euclidean distance* (in coordinate space) from the query point to every stored location, and returns the label of the closest one.

9.2 A Simple Geocoder — NapierLoc IV

Scalability Warning

Both methods have time complexity $\Theta(n)$ — that is, proportional to n (nu), the number of entries. Here n is small (three campuses), so this is fine. For large datasets, such as the entire UK postcode database (around 1.7 million entries), a hierarchical spatial data structure (e.g., a k-d tree) would be needed to keep searches fast. Θ (Theta, meaning "exact order of growth") is used to say the running time grows at exactly this rate — not faster, not slower.

9.2 Web-based Geocoding — Nominatim I

The simple `NapierLoc` geocoder only knows three locations. For real delivery problems we need a geocoder that can handle *any* valid UK address. The solution is to connect to a *web-based geocoding service*.

What a Web Geocoding Service Provides

- **Coverage**: handles millions of addresses across entire countries or the world.
- **Parsing**: interprets free-form address strings entered by end users, even with typos or abbreviated forms.
- **Updating**: the service provider keeps the data current; the application developer does not need to maintain any geographic database.

Nominatim is the official geocoding service for OpenStreetMap (OSM). It is free to use and is accessed via a simple HTTP request. The URL format for a forward geocoding query is:

```
https://nominatim.openstreetmap.org/search?q={address}&format=xml
```

9.2 Web-based Geocoding — Nominatim II

where {`address`} is a URL-encoded address string. Nominatim returns an XML document containing one or more matching places.

Reading the Nominatim XML Response

The XML response looks like this (simplified):

Nominatim API: Example XML Response for Address Lookup

```
<searchresults>
  <place
    place_id="282736548"
    lat="55.9329"
    lon="-3.2139"
    display_name="Napier University, Merchiston"
    type="university"
    importance="0.521"/>
</searchresults>
```

9.2 Web-based Geocoding — Nominatim IV

The key fields are:

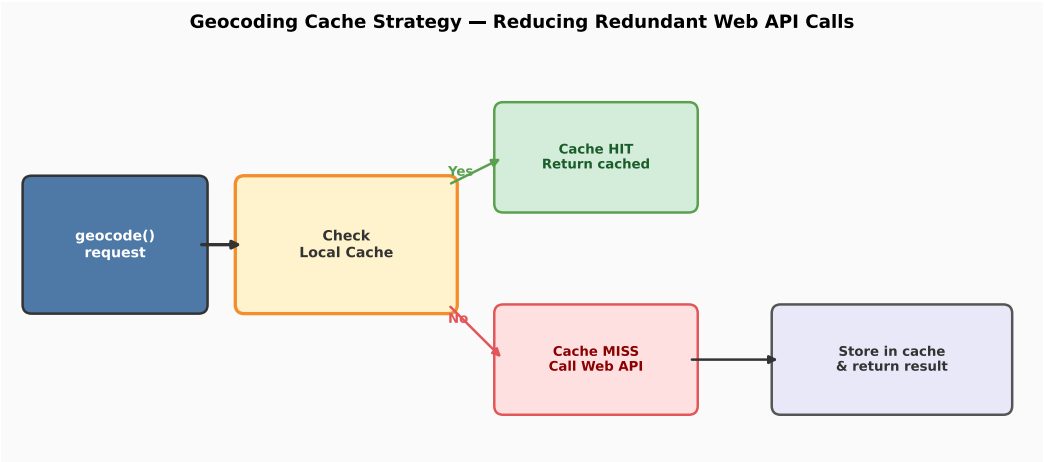
- `lat` and `lon`: the decimal-degree coordinates of the matched location.
- `display_name`: a human-readable description of the match.
- `importance`: a score (0 to 1) reflecting how prominent this place is; higher-importance matches should be preferred when multiple results are returned.
- `type`: the category of place (e.g., "university", "road", "city").

The application code must parse this XML (using a suitable library), extract the first result's `lat` and `lon` values, and return them as a `LatLon` object.

Caching Geocoding Requests — The Cache Class

A freely provided web service such as Nominatim imposes *usage limits*. Nominatim's policy (as of 2022) allows at most one request per second from any single client. In an optimisation experiment that calls the geocoder hundreds or thousands of times, this could be a bottleneck. More importantly, repeatedly geocoding the same address wastes network bandwidth and may violate the service's fair-use policy. The *Cache* design pattern solves this problem. The `Cache` class wraps any `Geocoder` and stores results in memory. When a request arrives, the cache first checks its stored results:

9.2 Web-based Geocoding — Nominatim VI



How the Cache Works (Detail)

- **Cache hit**: if the requested label is already in the internal list, return the stored coordinates immediately. No web request is made.
- **Cache miss**: if the label is not in the cache, delegate to the wrapped `Geocoder` (e.g., Nominatim), get the result, store it in the cache, and return it.

For reverse geocoding, the cache uses a *distance threshold*: if a stored point is within `REVERSE_THRESHOLD` degrees of the query point (set to `0.01` in the book's code), the cached label is returned. Otherwise, the web service is consulted.

The Cache as a Geocoder Decorator

The `Cache` class itself implements the `Geocoder` interface. The rest of the application sees it as just another geocoder. This is the *Decorator* design pattern: behaviour is added (caching) without modifying the original class. To use the cache: `Geocoder gc = new Cache(new Nominatim());`

9.2 Web-based Geocoding — Nominatim VIII

Key Insight

Caching is especially important during development and testing. When experimenting with an optimisation algorithm, the same set of addresses is geocoded repeatedly across many runs. A cache means the web service is contacted only once per unique address for the entire session.

9.2 Web-based Geocoding — Nominatim IX

Hosting Your Own Nominatim Instance

For large-scale projects — such as a logistics company running thousands of experiments per day — connecting to the public Nominatim service is impractical due to rate limits. An alternative is to *download and host Nominatim locally* on the developer's own hardware.

- Nominatim is open-source software that can be installed on any Linux server.
- OSM data for a country or region can be downloaded as a `.osm.pbf` file (typically a few gigabytes for the UK).
- Once installed, a local Nominatim instance has no rate limits and does not require an internet connection at query time.

This approach provides:

1. **Speed**: local queries take milliseconds rather than the 100–500 ms typical of a public API.
2. **Reliability**: no dependency on third-party uptime.
3. **Privacy**: address data never leaves the organisation's network.

Python: Geocoding with geopy + Nominatim

The Python geopy library provides a clean interface to Nominatim (and many other geocoding services) without writing raw HTTP requests. The example below geocodes two Edinburgh addresses, prints their coordinates, and then reverse-geocodes the first result to verify.

Listing 1: Forward and reverse geocoding using geopy

```
from geopy.geocoders import Nominatim
from geopy.extra.rate_limiter import RateLimiter
import time

# Create a geolocator.  The user_agent string identifies your application
# to Nominatim -- it must be unique and not "test" or "myapp".
geolocator = Nominatim(user_agent="delivery_optimiser_v1")

# RateLimiter wraps the geocode function to enforce a 1-second pause
# between requests, respecting Nominatim's usage policy.
geocode = RateLimiter(geolocator.geocode, min_delay_seconds=1)

# --- Forward geocoding: address string -> (latitude, longitude) ---
addresses = [
    "10 Colinton Road, Edinburgh",
    "Scottish Vintage Bus Museum, Fife",
]

locations = {}
for addr in addresses:
    loc = geocode(addr)
```

Python: Building a Simple In-memory Geocoder Cache

The following Python class mirrors the book's Java `Cache` design. It wraps any geopy geolocator and stores results in a dictionary, so each unique address is only sent to the web service once.

Listing 2: A simple geocoding cache in Python

```python
from geopy.geocoders import Nominatim
from geopy.extra.rate_limiter import RateLimiter
import math

class GeocoderCache:
    """
    Wraps a geopy geolocator with an in-memory cache.
    Forward cache: { address_string -> (lat, lon) }
    Reverse cache: list of ( (lat, lon), label ) pairs
    """
    REVERSE_THRESHOLD = 0.01  # degrees; ~1 km at UK latitudes

    def __init__(self, geolocator, min_delay=1.0):
        self._geocode = RateLimiter(
            geolocator.geocode, min_delay_seconds=min_delay
        )
        self._reverse = RateLimiter(
            geolocator.reverse, min_delay_seconds=min_delay
        )
        self._fwd_cache = {}        # address -> (lat, lon)
        self._rev_cache = []        # list of ((lat,lon), label)

```

9.3 Routing Services — Road-network Distances I

Why straight-line distances are not enough

A common shortcut in early implementations of routing algorithms is to use the *Haversine formula* to compute straight-line ("as the crow flies") distances between coordinates. While simple, this leads to inaccurate results in practice:

- Roads follow bends, hills, and rivers — they are never perfectly straight.
- A short straight-line distance may correspond to a very long drive if a river or motorway junction separates the two points.
- Journey *time* depends on speed limits, traffic signals, and traffic conditions, not just distance.

9.3 Routing Services — Road-network Distances II

A *routing service* computes distances and travel times along the actual road network, using graph search algorithms (typically variants of Dijkstra's algorithm or A*) applied to a road-network graph derived from OSM data.

What a Routing Service Returns (a Journey object)

For a given pair of start and end coordinates, a routing service returns:

- **Distance** (km): the road-network distance.
- **Travel time** (ms or minutes): the estimated journey duration.
- **Path**: a sequence of lat/lon waypoints defining the exact route to follow on the road network.

9.3 Routing Services — Road-network Distances III

The RoutingEngine Abstract Class

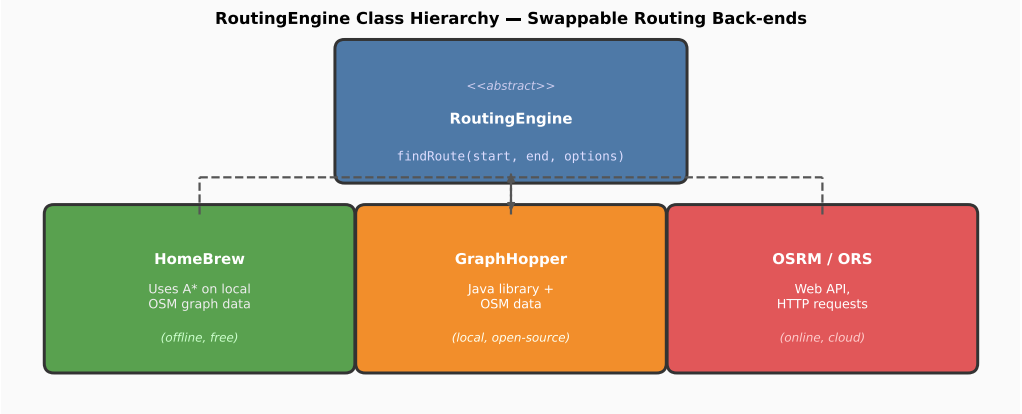
Just as the `Geocoder` interface made geocoders swappable, the book defines a `RoutingEngine` abstract class so that the choice of routing back-end (local library vs. web API) does not affect the rest of the application.
The single abstract method is:

```
Journey findRoute(LatLon start, LatLon end, HashMap<String,String> options)
```

The `options` map allows passing engine-specific parameters (such as vehicle type or data file location) without changing the method signature. Three concrete options keys are defined:

- `OPTION_DATA_DIR`: directory containing the OSM road data file.
- `OPTION_OSM_FILE`: the filename of the `.osm` data file.
- `OPTION_MODE`: travel mode, e.g., `"car"`, `"bike"`, `"foot"`.

9.3 Routing Services — Road-network Distances IV



9.3 Routing Services — Road-network Distances V

Three Routing Back-ends

The book implements and compares three concrete routing engines:

1. **HomeBrew**: a custom Java implementation that reads an OSM XML file, builds an in-memory road graph, and applies the A* algorithm to find the shortest path. It is the most instructive to study (the book's Chapter 8 builds it from scratch) but the slowest in practice.
2. **GraphHopper**: an open-source Java routing library built on top of OSM data. It is much faster than HomeBrew because it pre-processes the road graph for speed. GraphHopper supports multiple travel modes (car, bike, foot) and can run entirely offline.
3. **OSRM** (Open Source Routing Machine) or **OpenRouteService (ORS)**: web APIs that accept HTTP requests and return JSON responses. These are suitable when the client machine does not have enough RAM to load a full country-level road graph, or when a reliable internet connection is available.

The key advantage of the abstract class design is that all three engines produce the same `Journey` object. The optimisation algorithm is completely unaware of which engine is in use.

GraphHopper in Practice

GraphHopper (GH) is recommended for most research use-cases because it:

- Works offline using a downloaded OSM file (no API keys required).
- Handles a full country-level road graph in a few hundred MB of RAM.
- Returns results in milliseconds, fast enough for optimisation experiments that call `findRoute()` millions of times.

When first called, the GH implementation checks whether the road graph has already been initialised. If not, it loads the OSM file and builds the routing graph (a one-time cost taking a few seconds). Subsequent calls reuse the loaded graph.

Worked Example: Edinburgh to Glasgow by Car

Using a Scotland OSM file and GraphHopper in car mode:

- Start: 55.9329N, 3.2139W (Napier University, Edinburgh)
- End: 55.8617N, 4.2583W (University of Glasgow)
- Road distance: ≈ 74 km
- Estimated travel time: ≈ 65 minutes (by car, no traffic)
- Path: a sequence of $\sim$250 waypoints tracing the M8 motorway

A straight-line calculation gives only ≈ 65 km, underestimating the road distance by about 12%.

9.3 Building the Distance Matrix I

For delivery optimisation, we need pairwise distances between all locations — not just one pair. If there are n (the number of locations) delivery stops plus one depot, there are $n(n+1)$ ordered pairs (since distance from A to B may differ from B to A if one-way streets exist).

Distance Matrix Definition

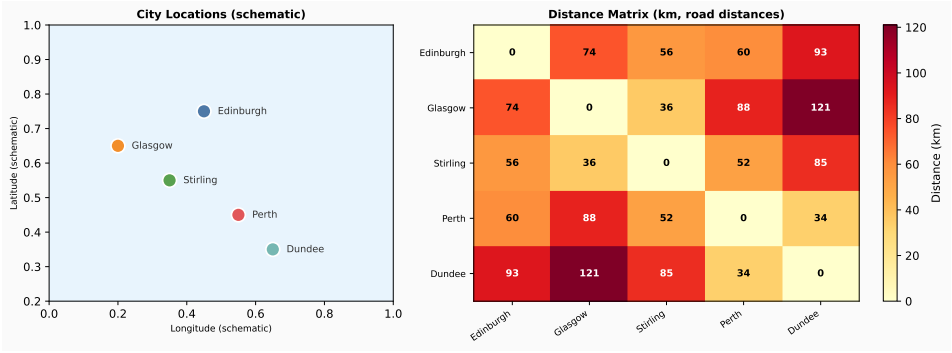
Let D be an $n \times n$ matrix where D_{ij} (read: "D sub i j") is the road-network travel distance (or time) from location i to location j. In most road networks:

$$D_{ij} \approx D_{ji}$$

but strict equality does not hold in general (one-way streets, turn penalties). The diagonal entries $D_{ii} = 0$ (distance from a location to itself is zero).

How to read this formula: D_{ij} means "the element in row i, column j of matrix D". The subscripts i and j are indices: i identifies the *origin* location and j the *destination* location. For a problem with $n = 5$ stops, D is a 5×5 table containing 25 entries.

9.3 Building the Distance Matrix II



9.3 Building the Distance Matrix III

Computing the Distance Matrix Efficiently

A naive approach calls `findRoute()` once for every ordered pair, which means n^2 routing queries. For $n = 100$ stops this is 10,000 queries — fine for an offline library, but potentially slow for a web API. Several web APIs (OSRM, OpenRouteService) accept a *matrix endpoint* that computes all n^2 pairs in a single HTTP request, much more efficiently than n^2 separate requests.

9.3 Building the Distance Matrix IV

Matrix Computation: 5 Scottish Cities

(km)	Edin.	Glasg.	Stirl.	Perth	Dundee
Edinburgh	0	74	56	60	93
Glasgow	74	0	36	88	121
Stirling	56	36	0	52	85
Perth	60	88	52	0	34
Dundee	93	121	85	34	0

Road distances (approximate) from OSRM. Note: the matrix is approximately symmetric but not exactly (e.g., motorway routing may differ in each direction).

Once the distance matrix is computed, it is stored in memory. The optimisation algorithm then looks up D_{ij} in O(1) time (constant time, meaning one memory access) rather than making a fresh routing request every time it evaluates a route.

9.3 Building the Distance Matrix V

Key Insight

Pre-computing and caching the full distance matrix is the single most important performance optimisation when using routing services. The matrix computation is a one-time cost; all subsequent evaluations by the optimisation algorithm are table lookups.

Python: Routing with OSRM Table API

The OSRM *table* endpoint computes a complete distance or duration matrix in one HTTP request. The example below builds a duration matrix (in seconds) for five Scottish cities using the public OSRM demo server.

Listing 3: Distance/duration matrix via OSRM Table API

```python
import requests
import numpy as np

# Coordinates: [longitude, latitude] format for OSRM
cities = {
    "Edinburgh": (55.9329, -3.2139),
    "Glasgow":   (55.8617, -4.2583),
    "Stirling":  (56.1165, -3.9369),
    "Perth":     (56.3964, -3.4370),
    "Dundee":    (56.4621, -2.9707),
}

names = list(cities.keys())
coords = list(cities.values())

# Build the OSRM coordinate string: lon,lat;lon,lat;...
coord_str = ";".join(f"{lon},{lat}" for lat, lon in coords)

# OSRM table API endpoint (public demo server -- replace with own instance
# for production; demo server has rate limits)
url = (f"http://router.project-osrm.org/table/v1/driving/{coord_str}"
```

Python: Routing with GraphHopper (Python Client)

If an internet connection is unavailable, GraphHopper can be run as a local server with a downloaded OSM file. The Python `graphh` package provides a convenient client.

Listing 4: Single route query with GraphHopper local server

```python
# pip install graphh
# Assumes GraphHopper is running locally: java -jar graphhopper.jar ...
import requests, json

GH_URL = "http://localhost:8989/route"   # local GraphHopper server

def get_route(lat1, lon1, lat2, lon2, vehicle="car"):
    """
    Query a locally running GraphHopper server for the route
    from (lat1, lon1) to (lat2, lon2).
    Returns: dict with 'distance_km' and 'time_min'.
    """
    params = {
        "point": [f"{lat1},{lon1}", f"{lat2},{lon2}"],
        "vehicle": vehicle,
        "locale": "en",
        "calc_points": "false",   # omit waypoints for speed
        "type": "json",
    }
    resp = requests.get(GH_URL, params=params, timeout=10)
    resp.raise_for_status()
    data = resp.json()
```

9.4 Exporting Data — Standard GIS Formats I

After the optimisation algorithm finds a good delivery route, the result must be communicated to humans and to other software systems. Three widely-used geospatial data formats make routes readable by mapping applications, GPS devices, and spreadsheets.

KML — Keyhole Markup Language

An XML-based format originally designed for Google Earth (OGC standard, 2015). KML is used to define *overlays* on top of a map: polygons, lines, markers, and labels. Route waypoints and the connecting path can be encoded as KML and then displayed using:

- Google Earth (desktop application)
- Leaflet or OpenLayers (web mapping libraries)
- Any online KML viewer

KML files can be served over HTTP and rendered in a browser in real time.

9.4 Exporting Data — Standard GIS Formats II

GPX — GPS Exchange Format

An XML-based format for GPS device data (track logs and waypoints). A GPX file contains *waypoints* (named points) and *tracks* (a sequence of coordinates forming a route). GPX files can be loaded directly into in-vehicle Sat Nav systems, phone navigation apps (e.g., OsmAnd, Google Maps), and GIS software.

CSV — Comma Separated Values

A plain-text tabular format. Each row is a waypoint; columns might include label, latitude, longitude, arrival time, and demand. CSV files can be opened in Excel, LibreOffice Calc, or imported into any database. They are the simplest format for exchanging data with business systems.

9.4 Exporting Data — Standard GIS Formats III

Export Format Comparison: KML vs GPX vs CSV

KML (Google Earth / Leaflet)

```
<Placemark>
  <name>Start</name>
  <Point>
    <coord>
     -3.21 55.93 0
    </coord>
  </Point>
</Placemark>
```

GPX (GPS / Sat Nav)

```
<gpx version="1.0">
  <trk>
   <trkseg>
    <trkpt
     lat="55.93"
     lon="-3.21"/>
   </trkseg>
  </trk>
```

CSV (Spreadsheet / Database)

```
label,lat,lon
Edinburgh,55.93,-3.21
Glasgow,55.86,-4.25
Stirling,56.12,-3.94
Perth,56.40,-3.47
Dundee,56.46,-2.97

(plain text table)
```

The three formats serve different audiences:

- **KML** for web mapping visualisation (planners, managers).
- **GPX** for drivers using GPS navigation devices.

9.4 Exporting Data — Standard GIS Formats IV

- **CSV** for data analysts and business system integration.

Key Insight

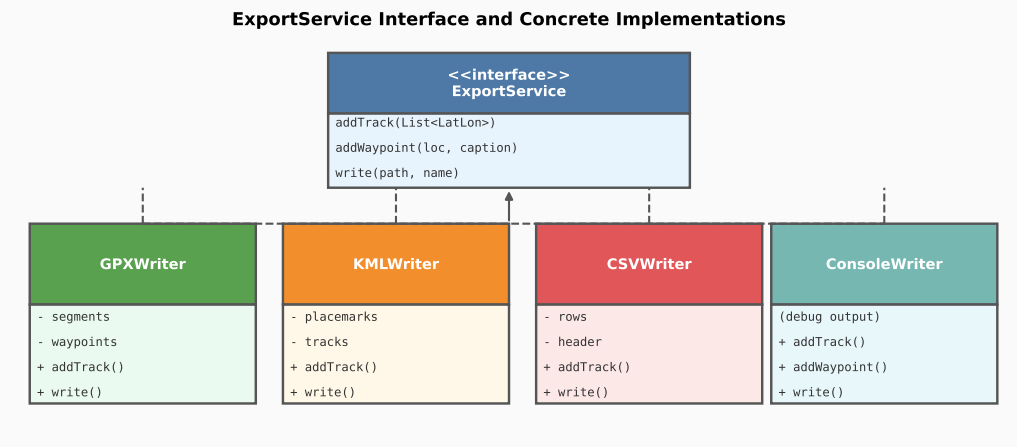
Supporting multiple export formats does not require rewriting the export logic from scratch for each format. The *ExportService* interface pattern (next frame) means that the same route data can be written to all three formats with a single call to a factory method.

9.4 The ExportService Interface I

Just as the `Geocoder` interface allowed geocoder swapping, the `ExportService` interface unifies all export formats behind a single API. The interface declares three operations:

- `addTrack(List<LatLon> locs)`: records a polyline (the route path) as a list of coordinates.
- `addWaypoint(LatLon loc, String caption, String desc)`: records a named point of interest (e.g., a delivery address).
- `write(String path, String name)`: writes the collected data to a file at the given path.

9.4 The ExportService Interface II



9.4 The ExportService Interface III

Four concrete implementations are provided in the book: `GPXWriter`, `KMLWriter`, `CSVWriter`, and `ConsoleWriter` (for debugging). All four implement `ExportService` so they can be used interchangeably.

9.4 The ExportService Interface IV

How the GPX Writer Works

A GPX file has the following overall structure:

1. An XML header identifying the GPX version and namespaces.
2. Zero or more `<wpt>` (waypoint) elements, one per delivery stop.
3. Zero or more `<trk>` (track) elements, each containing a `<trkseg>` (track segment) with a list of `<trkpt>` (track point) elements.
4. An XML footer closing the root `<gpx>` element.

The `GPXWriter` accumulates waypoints and track segments as strings in memory. When `write()` is called, it concatenates the header, the waypoints, the track segments, and the footer into a single string and writes it to a file.

9.4 The ExportService Interface V

Complete Workflow: Route from Edinburgh to Fife

The following sequence (Listing 9.14 in the book) demonstrates the full pipeline in action:

1. Geocode `"10 Colinton Road, Edinburgh"` to get start coordinates.
2. Geocode `"Scottish Vintage Bus Museum, Fife"` to get end coordinates.
3. Call `GraphHopper.findRoute(start, end)` to get the road path.
4. Call `export.addWaypoint(start, "Start", "Edinburgh Napier")`.
5. Call `export.addWaypoint(end, "Destination", "Bus Museum")`.
6. Call `export.addTrack(route.getPath())`.
7. Call `export.write("out", "journey")`.

The resulting file can be loaded into any GPX viewer to display the exact road-network route on an OSM base map.

9.4 Visualising the Result on a Map I

Once the route is exported to KML or GPX, it can be overlaid on an OpenStreetMap base map using a web visualiser.
The figure below shows an example GPX output (from the book's `ExportTest` demo) visualised using `gpxvisualizer.com`. The red line traces the road-network route from Edinburgh Napier University to the Scottish Vintage Bus Museum in Fife, including the Forth Road Bridge.

9.4 Visualising the Result on a Map II


```
1   private class Entry{
2       /*
3        * An inner class used to store a mapping between a label
4        * and a location (stored as a Visit)
5        */
6       private LatLon location;
7
8       public LatLon getLocation() {
9           return location;
10      }
11
12      public String getLabel() {
13          return label;
14      }
15
16      private String label;
17
18      public Entry(LatLon loc, String label) {
19          this.location =loc;
20          this.label = label;
21      }
22  }
```

Listing 9.3 The inner class *Entry* used within *NapierLoc*

The difficulties in geocoding stem from the amount of data that must be available to the geocoder to look up. In our simple example, both the *geocode()* and *reverseGeocode()* methods have a complexity of θn so they won't scale up as

Python: Writing a KML File

The Python `simplekml` library makes it easy to create KML files for route visualisation. The following example creates a KML file with two waypoints and a connecting line (representing a one-leg route).

Listing 5: Writing a KML file with simplekml

```python
# pip install simplekml
import simplekml

def write_kml(waypoints: list, track: list, out_path: str, name: str):
    """
    Write a KML file with waypoints and a route track.

    waypoints : list of (lat, lon, label, description) tuples
    track     : list of (lat, lon) tuples defining the route path
    out_path  : directory to save the file
    name      : filename stem (no extension)
    """
    kml = simplekml.Kml()

    # Add waypoints (placemarks)
    for lat, lon, label, desc in waypoints:
        pnt = kml.newpoint(name=label, description=desc)
        pnt.coords = [(lon, lat)]   # KML uses (lon, lat) order!
        pnt.style.iconstyle.color = simplekml.Color.red
        pnt.style.iconstyle.scale = 1.2

    # Add track as a LineString
```

Python: Writing a GPX File

The gpxpy library provides a Pythonic interface for creating and reading GPX files.

Listing 6: Writing a GPX file with gpxpy

```
# pip install gpxpy
import gpxpy
import gpxpy.gpx

def write_gpx(waypoints: list, track: list, out_path: str, name: str):
    """
    Write a GPX file with named waypoints and a route track.

    waypoints : list of (lat, lon, name, description) tuples
    track     : list of (lat, lon) tuples
    """
    gpx = gpxpy.gpx.GPX()

    # Add waypoints
    for lat, lon, wpt_name, desc in waypoints:
        wpt = gpxpy.gpx.GPXWaypoint(
            latitude=lat, longitude=lon,
            name=wpt_name, description=desc
        )
        gpx.waypoints.append(wpt)

    # Add track segment
    gpx_track = gpxpy.gpx.GPXTrack(name=name)
    gpx.tracks.append(gpx_track)
```

Python: Writing a CSV File and Visualising with Folium

CSV export is the simplest format and is achieved with Python's built-in `csv` module. Folium wraps the Leaflet.js mapping library, allowing routes to be displayed on an interactive OSM base map saved as an HTML file.

Listing 7: CSV export and interactive map with Folium

```
import csv, os
# pip install folium
import folium

def write_csv(waypoints, out_path, name):
    """Write waypoints to a CSV file."""
    with open(f"{out_path}/{name}.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["label", "latitude", "longitude"])
        for lat, lon, label, *_ in waypoints:
            writer.writerow([label, lat, lon])
    print(f"Saved CSV: {out_path}/{name}.csv")

def write_folium_map(waypoints, track, out_path, name):
    """Create an interactive Leaflet map using Folium."""
    # Centre the map on the mean of all waypoints
    centre_lat = sum(w[0] for w in waypoints) / len(waypoints)
    centre_lon = sum(w[1] for w in waypoints) / len(waypoints)

    m = folium.Map(location=[centre_lat, centre_lon], zoom_start=10,
                   tiles="OpenStreetMap")
```

9.5 Conclusions I

This chapter has shown how a nature-inspired optimisation algorithm can be connected to real-world geographic data sources and services. The key accomplishment is bridging the gap between the abstract mathematical world of optimisation (where locations are just indices 0 to $n-1$ in a distance matrix) and the physical world (where locations have addresses, coordinates, and road connections).

9.5 Conclusions II

Summary of Tools and Their Roles

Tool	Purpose	Open Alternatives
Nominatim	Forward/reverse geocoding	Google Maps API
GraphHopper	Local routing (offline)	OSRM, Valhalla
OSRM / ORS	Web routing API	Google Directions API
KML writer	Map visualisation export	GeoJSON
GPX writer	GPS / Sat Nav export	TCX, FIT
CSV writer	Spreadsheet/DB export	JSON, Parquet
Folium	Interactive HTML maps	Leaflet.js, MapLibre

9.5 Conclusions III

The Significance of OpenStreetMap

Possibly the most important development enabling this chapter is the growth of OpenStreetMap (OSM) as a freely available, community-maintained geographic database. Before OSM, access to high-quality road-network data required expensive commercial licences that put real-world routing out of reach for academic researchers and small companies. OSM changed this fundamentally.
Building on OSM, the following open-source ecosystem has emerged:

- **Nominatim**: free geocoding for any address in OSM.
- **GraphHopper** and **OSRM**: fast local routing engines built on OSM road data.
- **Leaflet**, **OpenLayers**: web mapping libraries that display OSM tiles in browsers.
- **Folium**: a Python library wrapping Leaflet for rapid prototyping.
- **QGIS**: a full open-source GIS desktop application for spatial analysis.

9.5 Conclusions IV

Key Takeaway

The design patterns used in this chapter — interfaces for geocoders and routing engines, caching decorators, and a unified export service — make it straightforward to swap one provider for another as requirements change. A solver built with these patterns can be tested with a small, hard-coded geocoder and then deployed against Nominatim or Google Maps with minimal code changes.

9.5 Conclusions V

References

- Haklay, M., and P. Weber. 2008. OpenStreetMap: user-generated street maps. *IEEE Pervasive Computing* 7(4): 12–18.
- Nominatim. 2021. `https://nominatim.org/`
- GraphHopper GmbH. 2021. GraphHopper Directions API. `https://github.com/graphhopper/graphhopper`
- OGC. 2015. KML 2.3. `http://docs.opengeospatial.org/is/12-007r2/12-007r2.html`
- Overview — Nominatim Documentation. 2020. `https://nominatim.org/release-docs/develop/api/Overview/`
- Vladimir Agafonkin. 2020. Leaflet (open-source mapping library).
- Foundation, O.K. 2021. Global Open Data Index: Locations. `https://index.okfn.org/dataset/postcodes/`
- Google Maps. 2021. `https://www.google.com/maps/`

Python: Full End-to-End Pipeline

The following script ties together geocoding, routing, and export into a single demonstration. It geocodes two addresses, queries OSRM for the route, and writes KML and CSV output files.

Listing 8: End-to-end: geocode + route + export

```
from geopy.geocoders import Nominatim
from geopy.extra.rate_limiter import RateLimiter
import requests, os

geolocator = Nominatim(user_agent="ch9_demo_pipeline")
geocode = RateLimiter(geolocator.geocode, min_delay_seconds=1)

# 1. Geocode two addresses
start_addr = "10 Colinton Road, Edinburgh"
end_addr   = "Scottish Vintage Bus Museum, Fife"

start_loc = geocode(start_addr)
end_loc   = geocode(end_addr)
s = (start_loc.latitude, start_loc.longitude)
e = (end_loc.latitude,   end_loc.longitude)
print(f"Start: {s}   End: {e}")

# 2. Get route from OSRM (single route, with geometry)
coord_str = f"{s[1]},{s[0]};{e[1]},{e[0]}"
url = (f"http://router.project-osrm.org/route/v1/driving/{coord_str}"
       f"?overview=full&geometries=geojson")
data = requests.get(url, timeout=15).json()
```

Python: OpenRouteService (ORS) — Directions API

OpenRouteService (ORS) is a free alternative to OSRM with an easy-to-use Python client. An API key is required (free tier available).

Listing 9: Routing with OpenRouteService Python client

```python
# pip install openrouteservice
import openrouteservice as ors

# Replace with your ORS API key from https://openrouteservice.org/
client = ors.Client(key="YOUR_API_KEY_HERE")

coords = [
    [-3.2139, 55.9329],   # Edinburgh Napier  (lon, lat)
    [-3.9369, 56.1165],   # Stirling          (lon, lat)
    [-3.4370, 56.3964],   # Perth             (lon, lat)
    [-2.9707, 56.4621],   # Dundee            (lon, lat)
]

# Request a route visiting all coords in order (car driving)
route = client.directions(
    coordinates=coords,
    profile="driving-car",
    format="geojson",
)

# Extract summary
summary = route["features"][0]["properties"]["summary"]
```

Summary: Chapter 9 Key Points I

This chapter covered the three pillars of connecting optimisation to the real world. Here is a concise summary of what was learned:

Geocoding (Section 9.2)

- Geocoding converts addresses to (lat, lon) coordinates; reverse geocoding does the opposite.
- Nominatim (free, OSM-based) is a practical choice for research and small-scale deployment.
- Caching is essential to avoid redundant web requests and to respect rate limits.
- The `Geocoder` interface allows any geocoder to be swapped without changing the optimiser code.

Routing Services (Section 9.3)

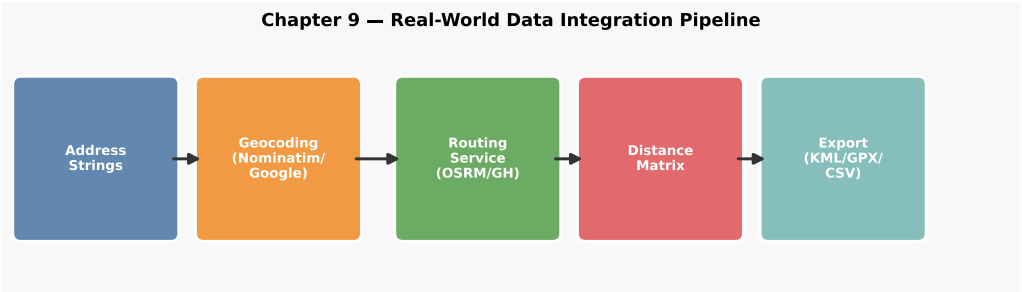
- Routing services compute actual road-network distances and travel times, which are significantly more accurate than straight-line approximations.
- The distance matrix (D_{ij}: distance from location i to j) should be pre-computed and cached before the optimiser runs.
- GraphHopper enables offline routing; OSRM and ORS provide web APIs.
- The `RoutingEngine` abstract class makes routing back-ends swappable.

Summary: Chapter 9 Key Points II

Exporting Data (Section 9.4)

- KML and GPX are XML-based geospatial formats for visualisation and navigation respectively.
- CSV is the simplest format for business system integration.
- The `ExportService` interface pattern allows writing to multiple formats with one common API.
- Folium (Python) creates interactive web maps from route data with minimal code.

Summary: Chapter 9 Key Points III



Summary: Chapter 9 Key Points IV

Final Takeaway

The emergence of OpenStreetMap and the open-source tools built on top of it has removed the geographic data barrier that once made real-world delivery optimisation inaccessible to researchers and small organisations. Combining a nature-inspired optimiser with Nominatim (geocoding), GraphHopper (routing), and Folium (visualisation) gives a complete, free, and practical solver for real delivery problems — with results that can be exported directly to a driver's GPS device.